



FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES

Departamento:

CIENCIAS DE LA COMPUTACION

Carrera: ELECTRONICA Y AUTOMATIZACION

Asignatura: FUNDAMENTOS DE PROGRHMACION
NRC: 28561

Apellidos y nombres del estudiante: DANIEL IZA

1. Leer una frase y contar cuántas letras y cuántos espacios tiene.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char frase[100];
```

```
    int letras = 0, espacios = 0, i;
```

```
    printf("Ingrese una frase: ");
```

```
    fgets(frase, sizeof(frase), stdin);
```

```
    for(i = 0; frase[i] != '\0'; i++) {
```

```
        if(frase[i] == ' ') {
```

```
            espacios++;
```

```
        } else if(frase[i] != '\n') {
```

```
            letras++;
```

```
        }
```

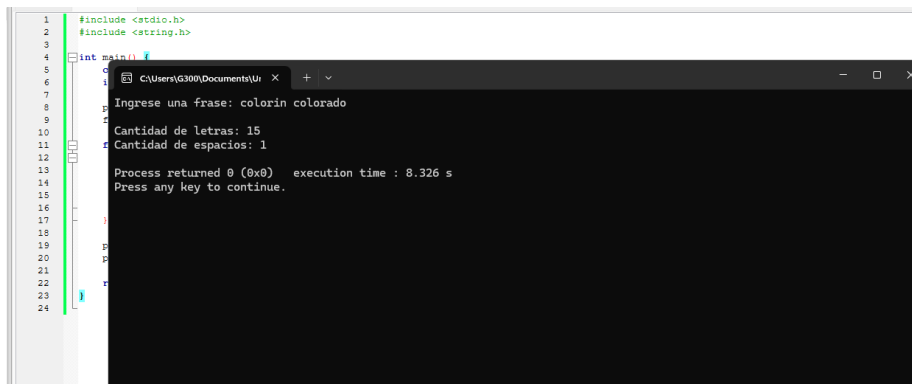
```
}
```

```
printf("\nCantidad de letras: %d", letras);
```

```
printf("\nCantidad de espacios: %d\n", espacios);
```

```
return 0;
```

```
}
```



The screenshot shows a C++ IDE with a code editor on the left and a console window on the right. The code in the editor is as follows:

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     char frase[100];
6     char espacios[100];
7     printf("Ingrese una frase: ");
8     fgets(frase, sizeof(frase), stdin);
9     int letras = 0;
10    int espacios = 0;
11    for (int i = 0; i < strlen(frase); i++) {
12        if (frase[i] != ' ') {
13            letras++;
14        } else {
15            espacios++;
16        }
17    }
18    printf("Cantidad de letras: %d\n", letras);
19    printf("Cantidad de espacios: %d\n", espacios);
20    return 0;
21 }
```

The console window shows the following output:

```
C:\Users\G300\Documents\Ur... x + -
Ingrese una frase: colorin colorado
Cantidad de letras: 15
Cantidad de espacios: 1
Process returned 0 (0x0)   execution time : 8.326 s
Press any key to continue.
```

2. Validar si una cadena es un palíndromo (ignorando espacios).

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
int main() {
```

```
    char frase[100], limpia[100];
```

```
    int i, j = 0;
```

```
    int esPalindromo = 1;
```

```
    printf("Ingrese una frase: ");
```

```
fgets(frase, sizeof(frase), stdin);
```

```
// Eliminar salto de línea y espacios
```

```
for(i = 0; frase[i] != '\0'; i++) {
```

```
    if(frase[i] != ' ' && frase[i] != '\n') {
```

```
        limpia[j] = tolower(frase[i]);
```

```
        j++;
```

```
    }
```

```
}
```

```
limpia[j] = '\0';
```

```
// Verificar palíndromo
```

```
int inicio = 0;
```

```
int fin = strlen(limpia) - 1;
```

```
while(inicio < fin) {
```

```
    if(limpia[inicio] != limpia[fin]) {
```

```
        esPalindromo = 0;
```

```
        break;
```

```
    }
```

```
    inicio++;
```

```
    fin--;
```

```
}
```

```

if(esPalindromo)

    printf("La cadena es un palindromo.\n");

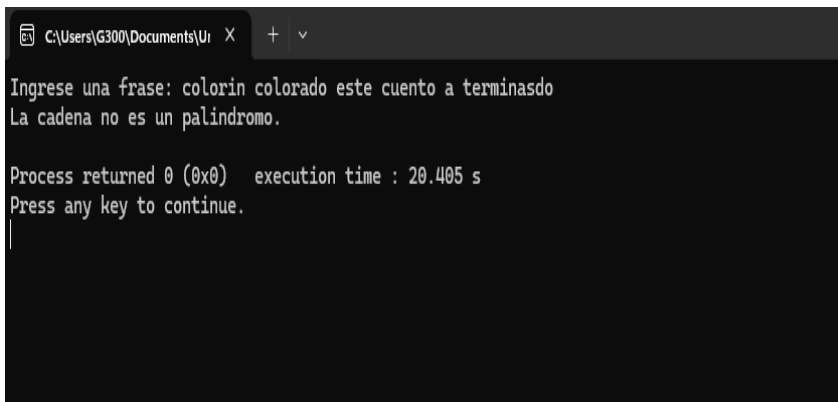
else

    printf("La cadena no es un palindromo.\n");


return 0;

}

```



```

C:\Users\G300\Documents\Ur  X + v
Ingrese una frase: colorin colorado este cuento a terminasdo
La cadena no es un palindromo.

Process returned 0 (0x0)   execution time : 20.405 s
Press any key to continue.
|

```

3. Unir nombre y apellido en una sola cadena con un espacio entre ambos.

```

#include <stdio.h>
#include <string.h>

int main() {
    char nombre[50];
    char apellido[50];
    char completo[100];

    printf("Ingrese el nombre: ");
    fgets(nombre, sizeof(nombre), stdin);
    nombre[strcspn(nombre, "\n")] = '\0';

    printf("Ingrese el apellido: ");
    fgets(apellido, sizeof(apellido), stdin);
    apellido[strcspn(apellido, "\n")] = '\0';

    strcpy(completo, nombre);
    strcat(completo, " ");
}

```

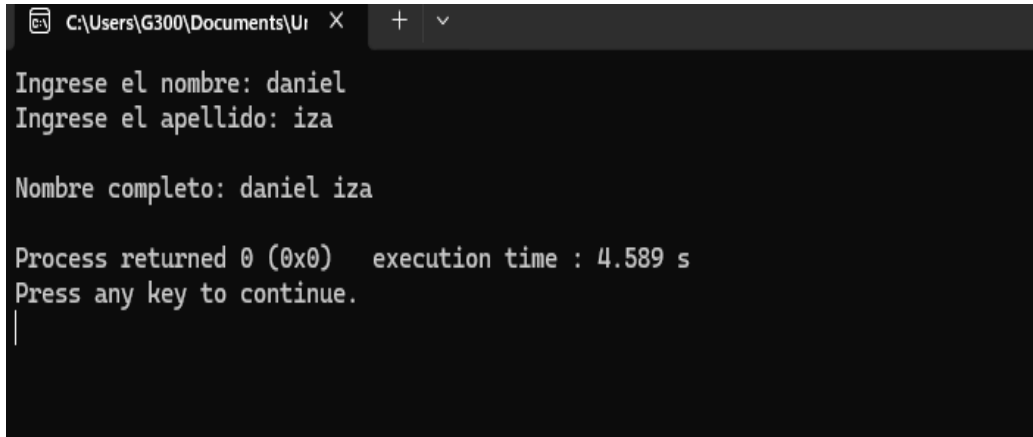
```

    strcat(completo, apellido);

    printf("\nNombre completo: %s\n", completo);

    return 0;
}

```



```

C:\Users\G300\Documents\U... X + v
Ingrese el nombre: daniel
Ingrese el apellido: iza

Nombre completo: daniel iza

Process returned 0 (0x0)   execution time : 4.589 s
Press any key to continue.
|

```

4. Buscar una palabra dentro de un texto e indicar la posición donde aparece por primera vez.

```

#include <stdio.h>

#include <string.h>

int main() {

    char texto[200];

    char palabra[50];

    char *posicion;

    printf("Ingrese un texto: ");

    fgets(texto, sizeof(texto), stdin);

    texto[strcspn(texto, "\n")] = '\0';

    printf("Ingrese la palabra a buscar: ");

```

```

fgets(palabra, sizeof(palabra), stdin);

palabra[strcspn(palabra, "\n")] = '\0';


posicion = strstr(texto, palabra);


if (posicion != NULL) {

    printf("La palabra se encontro en la posicion: %ld\n",

        posicion - texto);

} else {

    printf("La palabra no se encontro en el texto.\n");

}


return 0;

}

```

```

C:\Users\G300\Documents\U... X + v
Ingrese un texto: hola francisco como estas
Ingrese la palabra a buscar: francisco
La palabra se encontro en la posicion: 5

Process returned 0 (0x0) execution time : 23.878 s
Press any key to continue.

```

5. Convertir una cadena numérica a entero y calcular su cuadrado.

```

#include <stdio.h>

#include <stdlib.h>

```

```

int main() {

    char cadena[50];

    int numero, cuadrado;


    printf("Ingrese un numero: ");

    scanf("%s", cadena);


    numero = atoi(cadena);

    cuadrado = numero * numero;


    printf("El numero ingresado es: %d\n", numero);

    printf("Su cuadrado es: %d\n", cuadrado);


    return 0;

}

```

```

C:\Users\jesus\Documents> g++ 1.cpp
C:\Users\jesus\Documents> ./1.exe
Ingrese un numero: 5
El numero ingresado es: 5
Su cuadrado es: 25

Process returned 0 (0x0)   execution time : 1.124 s
Press any key to continue.

```

6. Invertir una cadena y mostrar la original y la invertida.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main() {
```

```
    char cadena[100];
```

```
    char invertida[100];
```

```
    int i, longitud;
```

```
    printf("Ingrese una cadena: ");
```

```
    fgets(cadena, sizeof(cadena), stdin);
```

```
    cadena[strcspn(cadena, "\n")] = '\0';
```

```
    longitud = strlen(cadena);
```

```
    for (i = 0; i < longitud; i++) {
```

```
        invertida[i] = cadena[longitud - 1 - i];
```

```
    }
```

```
    invertida[longitud] = '\0';
```

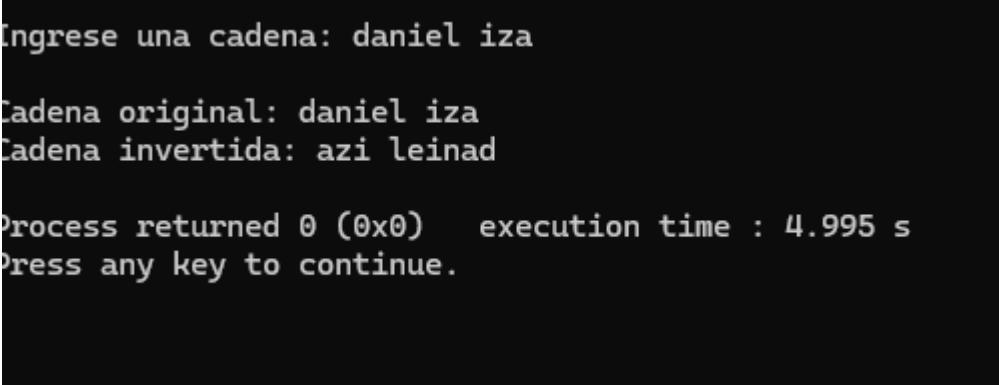
```
    printf("\nCadena original: %s\n", cadena);
```

```
    printf("Cadena invertida: %s\n", invertida);
```



```
return 0;

}
```



```
Ingrese una cadena: daniel iza
Cadena original: daniel iza
Cadena invertida: azi leinad
Process returned 0 (0x0)   execution time : 4.995 s
Press any key to continue.
```

7. . Crear una función que reciba una cadena y devuelva cuántas vocales tiene.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int contarVocales(char cadena[]) {
```

```
    int i, contador = 0;
```

```
    for(i = 0; cadena[i] != '\0'; i++) {
```

```
        if(cadena[i] == 'a' || cadena[i] == 'e' ||
```

```
           cadena[i] == 'i' || cadena[i] == 'o' ||
```

```
           cadena[i] == 'u' || cadena[i] == 'A' ||
```

```
           cadena[i] == 'E' || cadena[i] == 'I' ||
```

```
           cadena[i] == 'O' || cadena[i] == 'U') {
```

```
            contador++;
```

```
        }
```

```
    }
```

```

    return contador;
}

int main() {

    char cadena[100];

    printf("Ingrese una cadena: ");

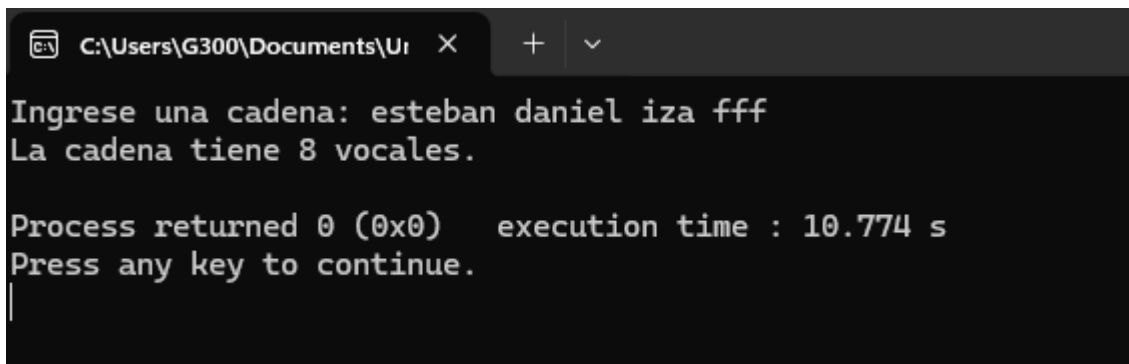
    fgets(cadena, sizeof(cadena), stdin);

    cadena[strcspn(cadena, "\n")] = '\0';

    printf("La cadena tiene %d vocales.\n", contarVocales(cadena));

    return 0;
}

```



```

C:\Users\G300\Documents\U... X + v
Ingrese una cadena: esteban daniel iza fff
La cadena tiene 8 vocales.

Process returned 0 (0x0)   execution time : 10.774 s
Press any key to continue.
|

```

ConclusionES

El uso de funciones en el manejo de cadenas en C permite resolver problemas de forma organizada y reutilizable, como el conteo de vocales.

Además, recorrer la cadena carácter por carácter ayuda a comprender y analizar su contenido de manera precisa y eficiente.

Recomendaciones

Se recomienda comprobar siempre la entrada de datos para evitar errores durante el procesamiento de cadenas.

También es conveniente practicar con frecuencia el uso de funciones y estructuras de control para mejorar el desarrollo de la lógica de programación.